Vector Embedding

❖ Summary: -

I created a system using Amazon Amazon Bedrock and a Lambda function that takes user text, converts it into vector data using an AI model, and returns the result. I also understood concepts like vectors, embeddings, data chunking, vector storage, and vector search used in AI applications.

❖ Vectors: -

- Why Do We Need Vectors: -
  - Computers don't understand text, images, or audio directly**.** They understand only numbers**.**
  - So we convert everything into vectors (arrays of numbers)**.**

- What are Vectors: -
  - Vectors are mathematical representation of words, sentences and documents.
  - Vectors are represented by list of numbers or sequence of numbers.

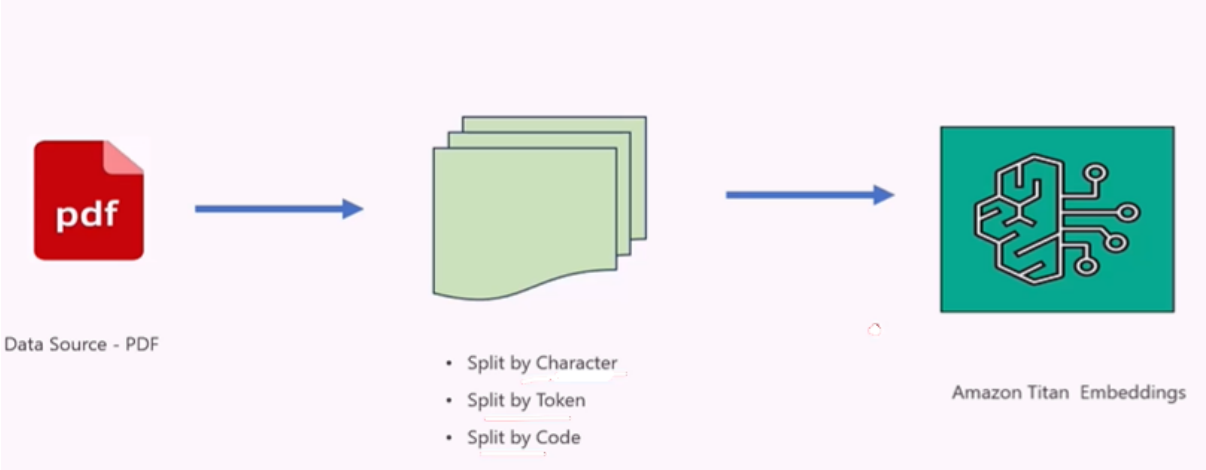
❖ Embedding Models: -

An embedding model is an AI model that transforms data into numerical vectors so that machines can understand and compare meaning.

Text / Image → Embedding Model → Vector (numbers)

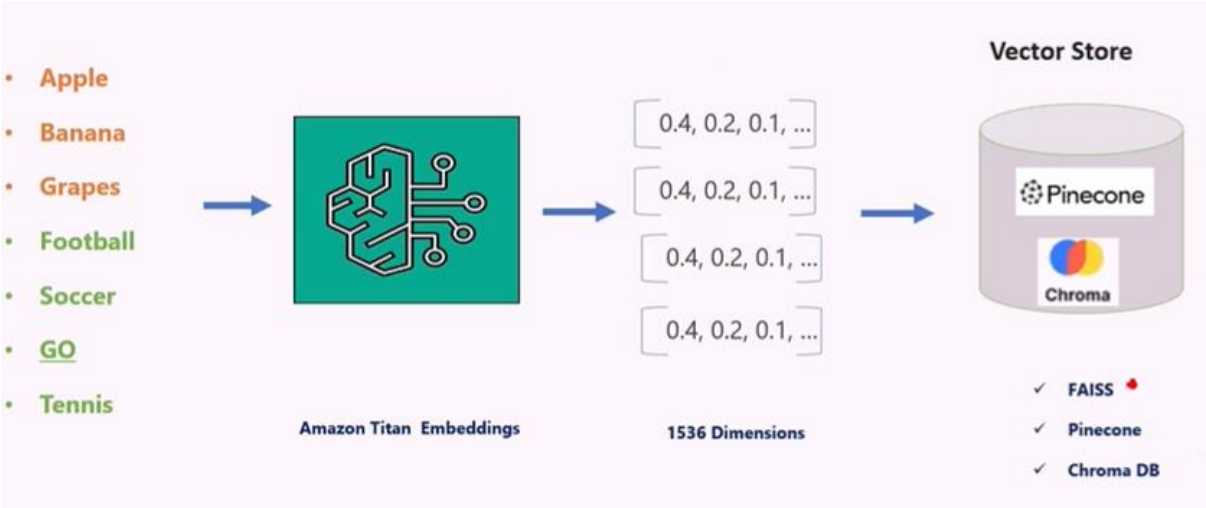
❖ Data Chunking: -

Data chunking is the process of dividing data into manageable pieces (chunks) so AI can process it efficiently.



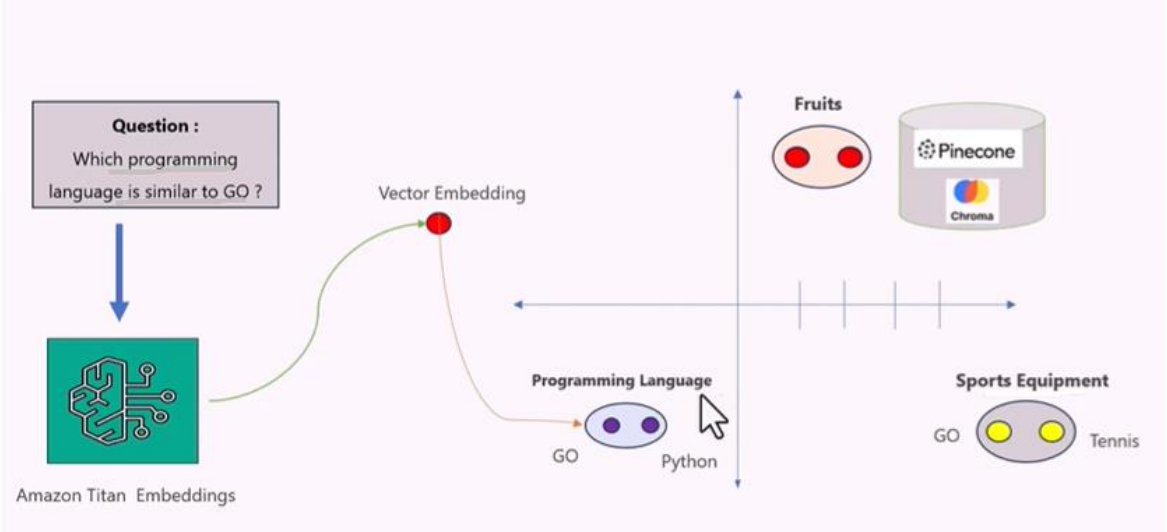
❖ Vectors Store: -

A vector store is a database that stores embedding's and enables fast similarity search for AI applications.



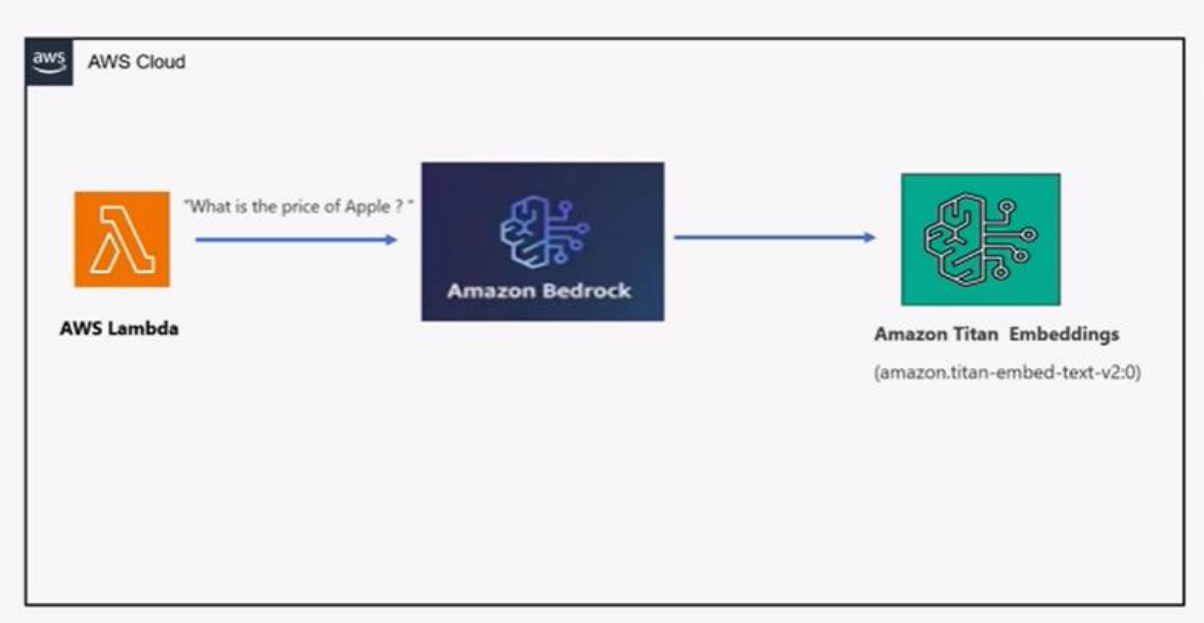
❖ Vector Search: -

Vector search is the process of finding similar items by comparing their vector representations using similarity measures.



❖ Vector and Embedding's In AWS Cloud: -

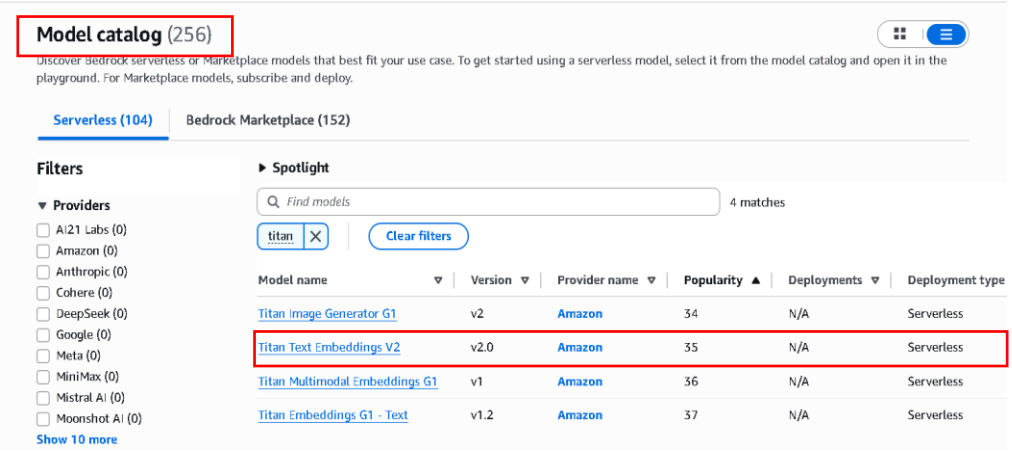
- So what we will do we have the AWS Cloud.



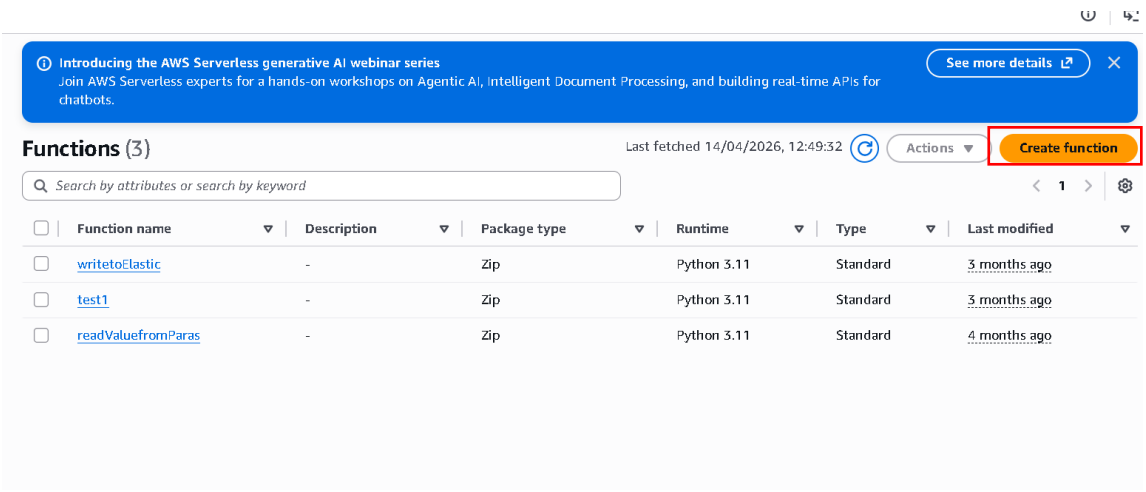
- With AWS cloud we create Lambda function.
- And through the lambda function we are going to send some prompts (What is the price of Apple?).
- Then we are going to send this word of sentences to Amazon Bedrock
- And Amazon bedrock behind the scenes is going to send this request to Amazon Titan Embedding's.
- And in particular we are going to use Amazon Dot titan Embed text version 2.0 Model.
- So once the request is going to Amazon titan embedding model.
- Its going to create a vector embedding of that particular model and send back the response to the lambda function.

❖ Now we are going to build this: -

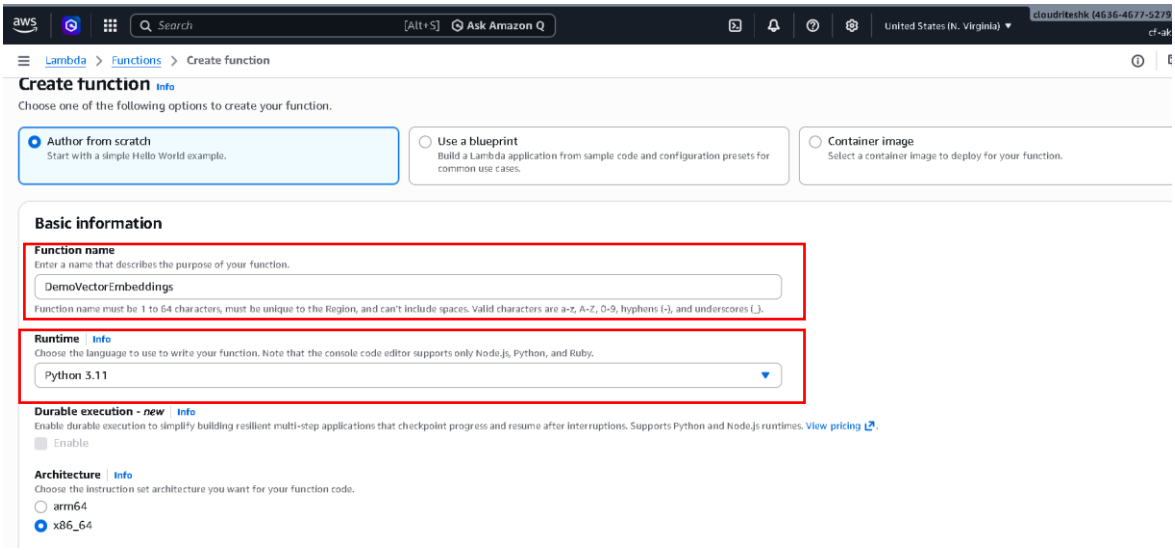
- Firstly go to your console and search Amazon Bedrock.
- Then go in view model catalog
- And search Titan Text Embeddings V2 in search bar.



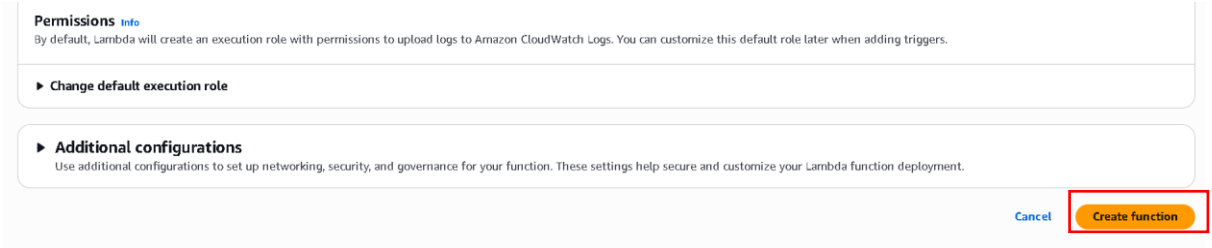
- Open this Titan Text Embeddings V2



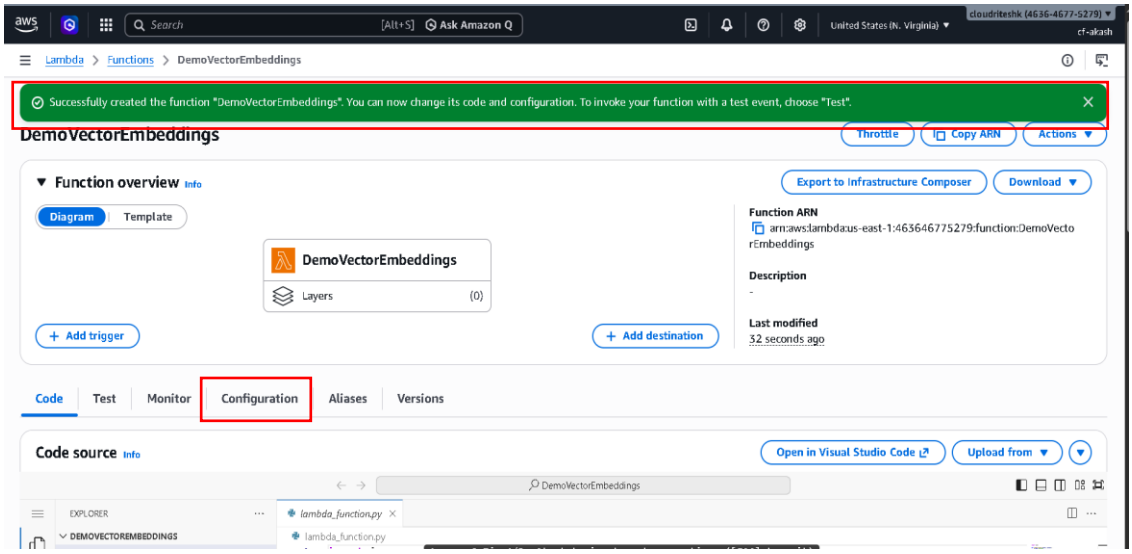
- Now open Another AWS console in new tab and Search Lambda in search bar.
- Click on the create function.



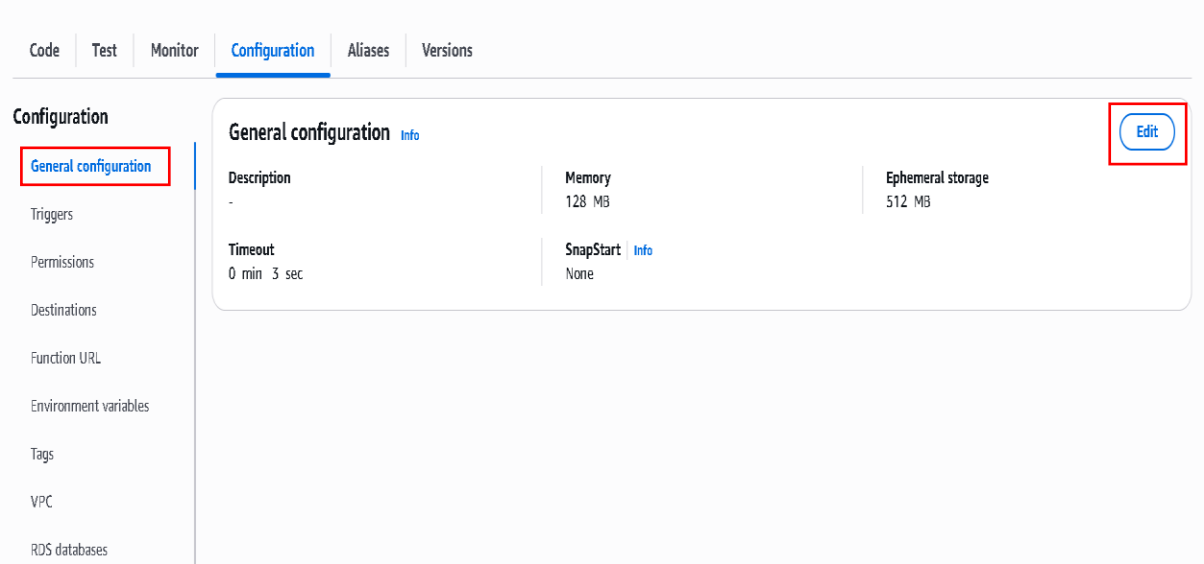
- choose author from scratch.
- Write function Name
- And choose Runtime (Programing language) I choose Python 3.11



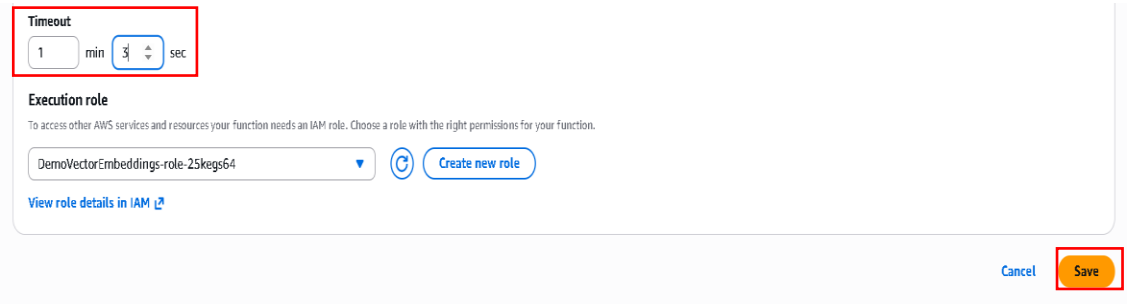
- Now click on create Account.



- So our lambda function is created.
- And now you go on configuration
- And in configuration go general configuration section.
- And click on Edit.

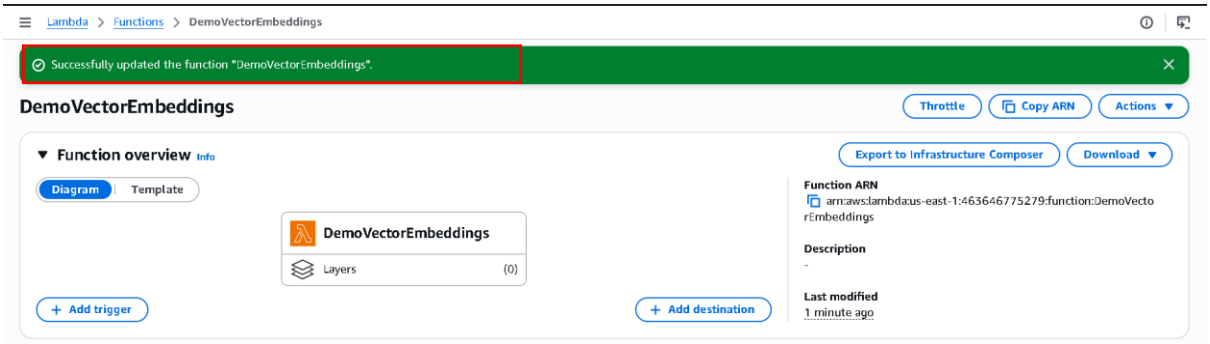


- And in configuration go general configuration section.
- And click on Edit.

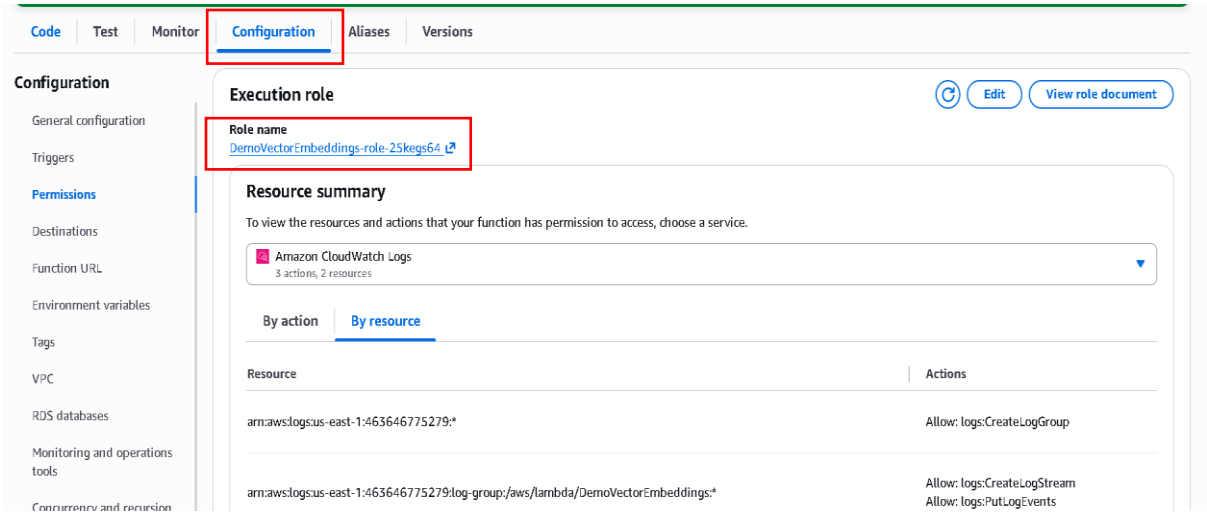


- The default timeout for lambda function is 3 second
- So I will make it 1 min 3 sec because it might have to do some processing.

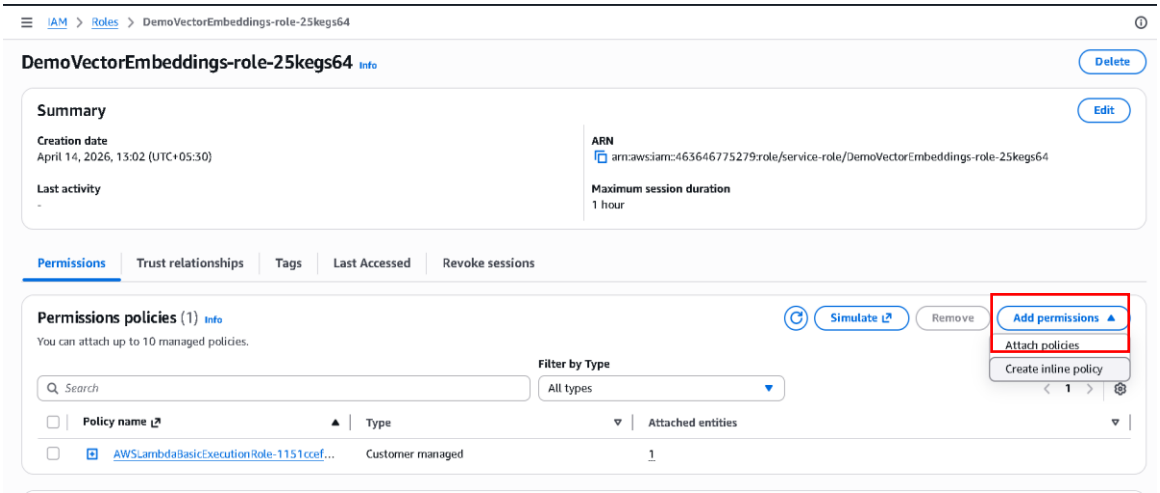
- And click on save.



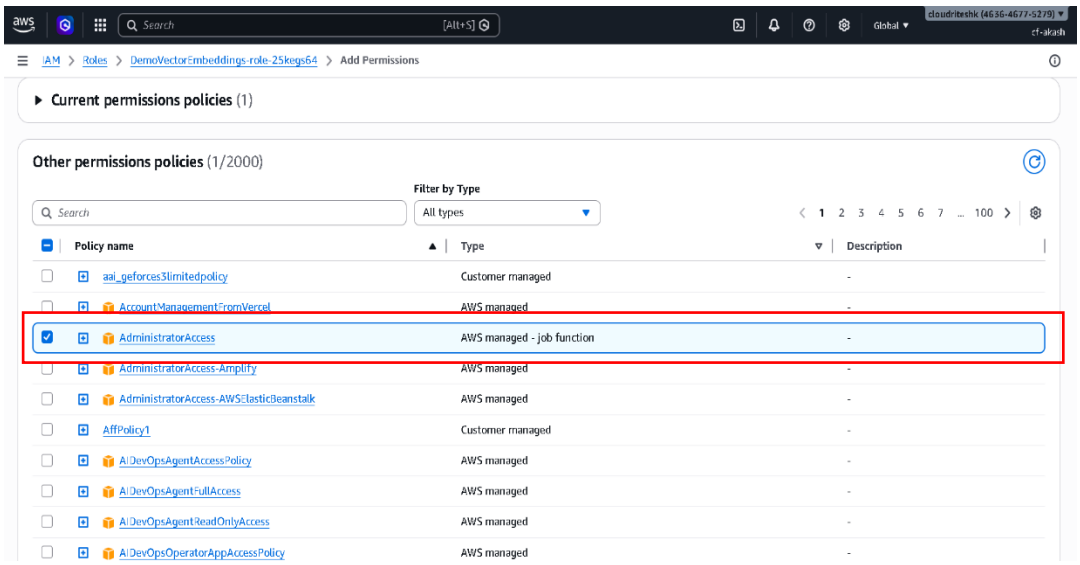
- Function is successfully updated.
- Now Lambda Function also requires access to Amazon Bedrock.



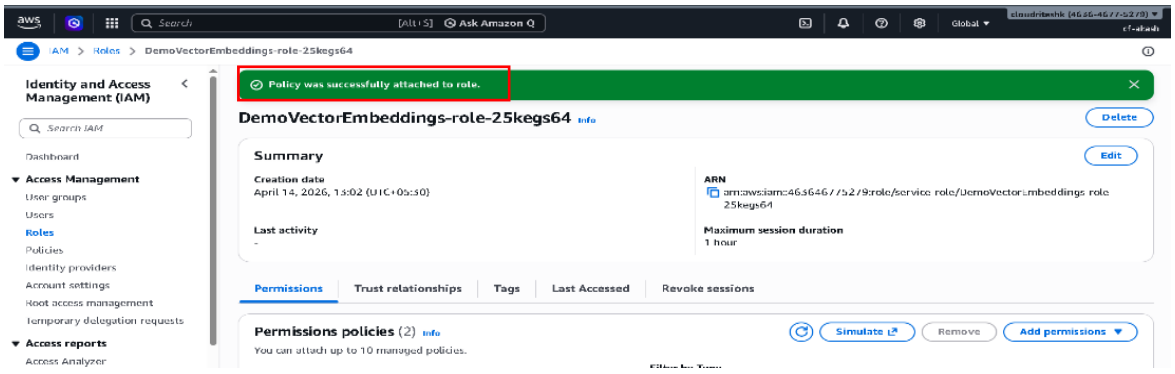
- So we'll give it the required permissions.
- So go to configuration and click on Permission
- Now click on role name
- And then in role name click on ADD Permissions



- In Add Permission click Attach Policies.



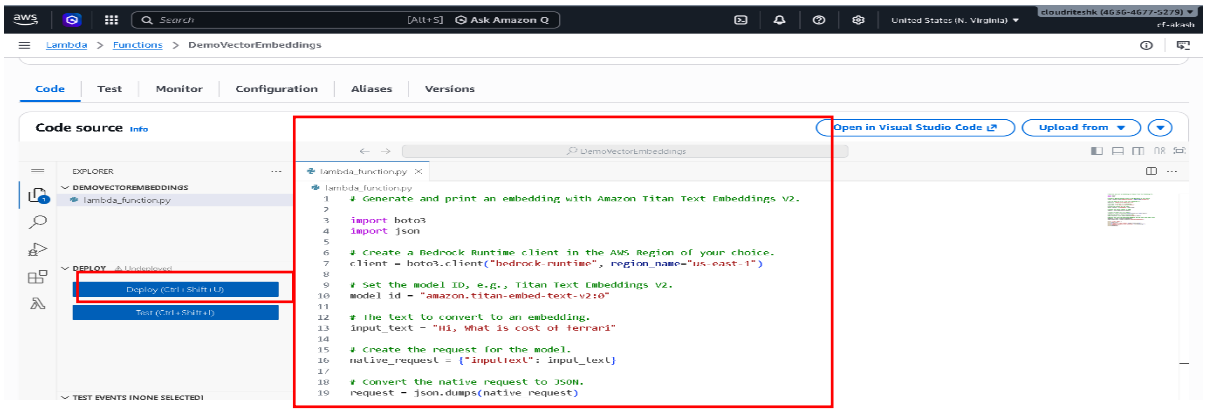
- So we'll give admin access
- This is not done in production Environment but for this demo we'll just give it admin access.
- Now just scroll down click on add Permissions.



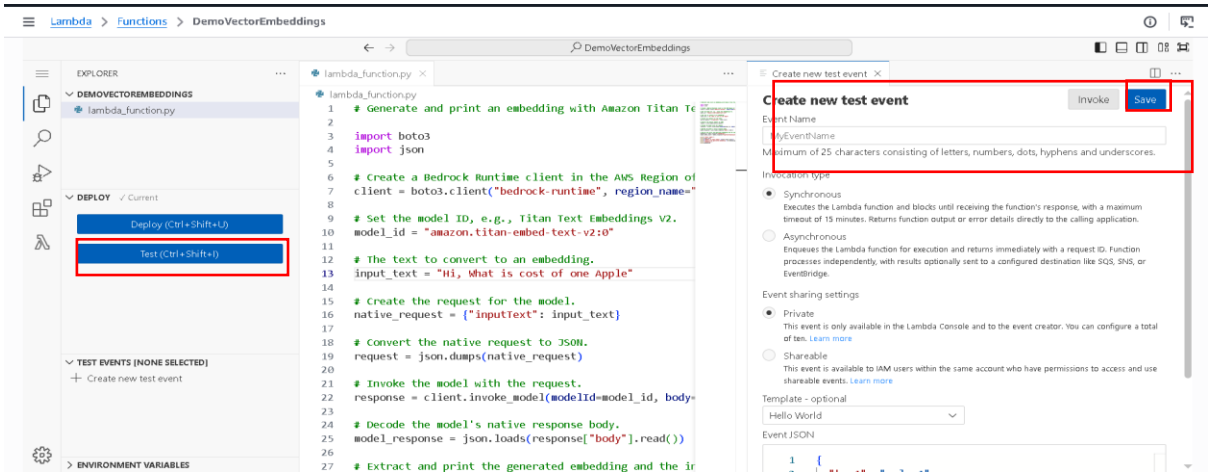
- Now Policy was successfully attached to role.
- Go back on lambda function tab and click on code.
- And paste the below code in the code section.

```python
import boto3
import json

def lambda_handler(event, context):

    # 🔹 Step 1: Create Bedrock client
    client = boto3.client("bedrock-runtime", region_name="us-east-1")

    # 🔹 Step 2: Model ID (Titan Embeddings V2)
    model_id = "amazon.titan-embed-text-v2:0"

    # 🔹 Step 3: Get input text (dynamic or default)
    input_text = event.get("text", "What is the price of apple?")

    # 🔹 Step 4: Create request
    request_body = {
        "inputText": input_text
    }

    # 🔹 Step 5: Convert to JSON
    request = json.dumps(request_body)

    # 🔹 Step 6: Call Bedrock model
    response = client.invoke_model(
        modelId=model_id,
        body=request
    )

    # 🔹 Step 7: Read response
    response_body = json.loads(response["body"].read())

    embedding = response_body["embedding"]
    token_count = response_body["inputTextTokenCount"]

    # 🔥 Step 8: Print logs (CloudWatch)
    print("===== DEBUG LOGS =====")
    print("Input Text:", input_text)
    print("Token Count:", token_count)
    print("Embedding Size:", len(embedding))
    print("First 10 Embedding Values:", embedding[:10])

    # 🔹 Step 9: Return response (avoid full embedding)
    return {
        "statusCode": 200,
        "body": {
            "input": input_text,
            "token_count": token_count,
            "embedding_size": len(embedding),
            "sample_embedding": embedding[:10]   # only sample
        }
```

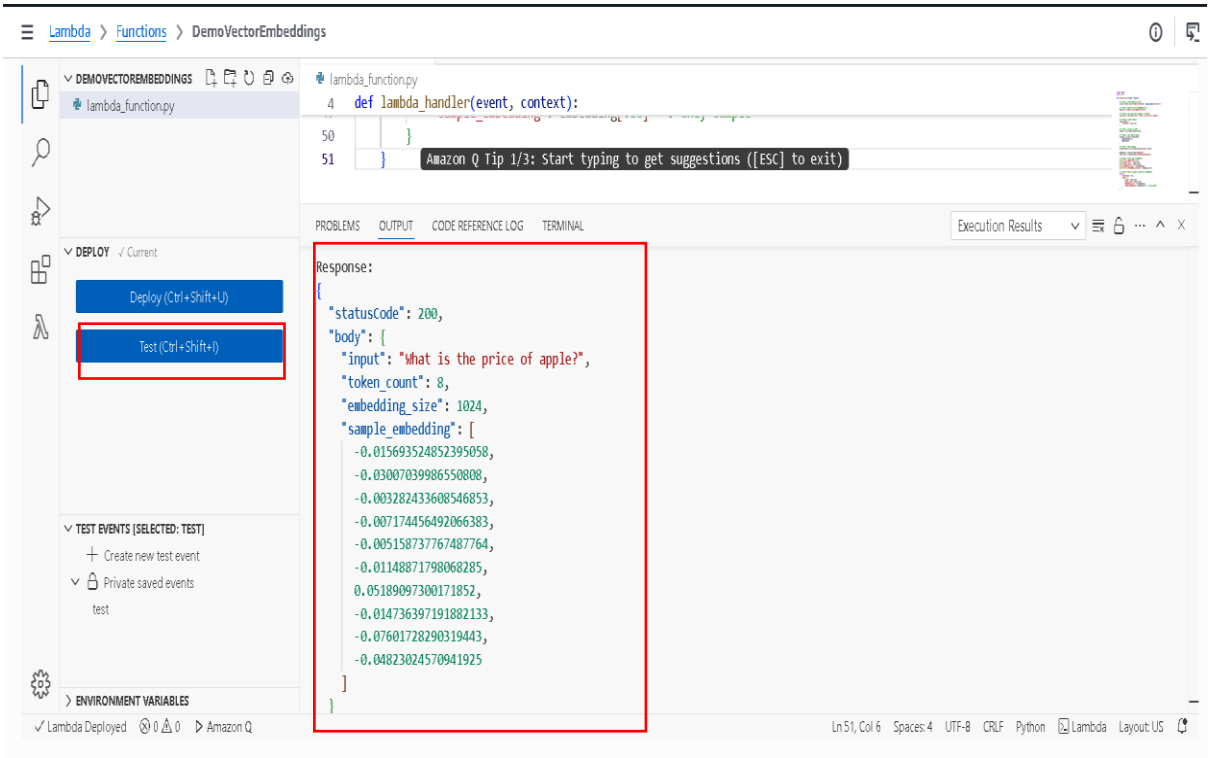
```
}
```



- Now click on Deploy
- Our code is deployed



- Now click on Test
- A popup window comes select create new test event
- Write event name (like test) and click on save.



- And then again click on test button.
- Your function runs and give output.